Student Grade Management System

Advanced Topics Guide — Collections · Exceptions · Concurrency · File I/O · Regex

Reviewer-defense prep & study guide for the v3 "Advanced Edition" + Backup & Restore work

Student Grade Management System — Advanced Topics Guide

A study guide and reviewer-defense prep for the v3 "Advanced Edition" + Backup & Restore work.

This document explains **what was built, where it lives, and why it was built that way** — grounded in the actual code in this repository, not in generic textbook descriptions. Every class named below is real; every design decision explained here is the one actually made in this codebase, including the places where the implementation deliberately diverges from the "obvious" textbook answer.

---

1. What "Advanced" Means Here

The original project brief (`REAME-V3.md`) asked for five capabilities:

1. Type-safe data structures using the **Collections Framework** and generics
2. Modern **file I/O** via NIO.2 and Stream processing, in multiple formats
3. **Regex**-based input validation
4. **Thread-safe concurrent** background operations via the Executor framework
5. Performance optimization via **caching** and correct concurrent access patterns

Everything below maps onto one of these five, plus one feature added afterward outside the original brief: a **Backup & Restore** capability that deliberately introduces this project's first *checked* exception, to demonstrate that pattern too.

---

2. New Folders & Files — Full Inventory

`main.concurrent` (new package — Sprint 2/3, PBI-5, 6, 7, 8, 9)

File	Role
`StatisticsDashboard.java`	Background thread that prints a live class-statistics snapshot every 5s until stopped.
`ScheduledGpaRecalculationJob.java`	Recalculates class-wide GPA rankings on a fixed daily schedule.
`BatchReportService.java`	Generates/exports grade reports for many students at once, one task per student on a thread pool.
`LruCache.java`	Generic, thread-safe, fixed-capacity cache with least-recently-used eviction.
`AuditTrail.java`	Durable, structured log of every add/update/delete, written asynchronously off the caller's thread.

`main.dataio` (new package — PBI-2, multi-format export/import)

File	Role
`StudentRecord.java` / `GradeRecord.java`	Flat, format-agnostic `record` snapshots of a `Student` / `Grade` — every field needed to reconstruct the original, independent of CSV/JSON/binary.
`StudentDataExporter.java` / `GradeDataExporter.java`	Write a `List< ... Record>` to CSV, JSON (Jackson), or Java binary serialization.
`StudentDataImporter.java` / `GradeDataImporter.java`	The inverse — read each format back into a `List< ... Record>`.
`StudentRecordMapper.java` / `GradeRecordMapper.java`	Convert between the domain object (`Student` / `Grade`) and its flat `Record` form, both directions.

`main.backup` (new package — Backup & Restore feature)

File	Role
`BackupManifest.java`	Small header record: format version, timestamp, student/grade counts.
`BackupPayload.java`	The full contents of one backup file: manifest + every `StudentRecord` / `GradeRecord`.
`BackupService.java`	Interface — `createBackup()` / `restoreBackup()`, both declaring `throws BackupException`.
`BackupServiceImpl.java`	Implementation: serializes/deserializes a `BackupPayload` to one JSON file via Jackson.

`main.exceptions` — new files

File	Role
`BackupErrorCode.java`	Enum: `IO_FAILURE`, `CORRUPT_FILE`, `VERSION_MISMATCH` — distinguishes *why* a backup failed.
`BackupException.java`	This project's one deliberately **checked** exception. See §5.

`main.utils.validators` (new package — PBI-3, regex validation)

File	Role
`ValidationPatterns.java`	Single source of truth for every regex: student/grade ID format, email, phone, course code, ISO date.
`StudentValidator.java` / `GradeValidator.java` / `SubjectValidator.java`	Validate a domain object's fields against those patterns, throwing a specific exception per failure.

`main.console` — new file

File	Role
`BackupAction.java`	Menu option 11: Backup & Restore — the one action whose collaborator throws a checked exception.
`AbstractMenuAction.java` / `AbstractGradeAction.java`	Shared base classes extracted later to eliminate duplicate `getOptionNumber()` / `getLabel()` /constructor boilerplate across every menu action.

`main.manager` — new file

File	Role
`StudentSearcher.java`	Regex/pattern-based search: by ID, partial name, grade range, type, or an arbitrary email regex.

Modified, not new

`GradeManager` / `StudentManager` gained a `ReadWriteLock`, an `LruCache`, an `AuditTrail`, and (for Backup & Restore) a `getAllGrades()` passthrough. `GradeRepositoryImpl` / `StudentRepositoryImpl` / `SubjectRepositoryImpl` were rewritten from fixed arrays to `Map`-backed storage. `FileExporter` was rewritten from `java.io.File` / `FileWriter` to NIO.2 (`Path` / `Files`).

---

3. Collections Framework — What's Used, Where, and Why

The brief specifically suggested `HashMap` (O(1) lookup), `TreeMap` (sorted rankings), and `HashSet` (unique tracking). Here's what actually happened, including where the code deliberately did **not** reach for the "obvious" answer:

`LinkedHashMap` instead of plain `HashMap`, everywhere lookups matter

`StudentRepositoryImpl`, `GradeRepositoryImpl`'s secondary index, and `SubjectRepositoryImpl` all use `LinkedHashMap<String, T>` keyed on ID/code, not a plain `HashMap`. Why the extra specificity? A plain `HashMap` gives no ordering guarantee — iterating it can return entries in any order, and that order can even change between runs. `LinkedHashMap` preserves **insertion order**, so `getAllStudents()` keeps returning students in the same order the old array-based version did. This mattered concretely: existing tests and console code grab "the first seeded student" via `getAllStudents().get(0)` — a plain `HashMap` would have silently broken that without any code change being wrong on its own.

```
// StudentRepositoryImpl.java
private final Map<String, Student> students = new LinkedHashMap<>();
```

This is the real lesson here: **the right collection isn't just "fast" — it's the one whose iteration/ordering contract matches what your callers actually depend on.**

`TreeMap` for sorted GPA rankings — exactly as specified

`GPACalculator.classRankings()` returns a `NavigableMap<Double, List<Student>>`, backed by a `TreeMap` with a reversed comparator, so the highest GPA sorts first automatically — no manual sort step, and the map *stays* sorted as you read it:

```
// GPACalculator.java
public NavigableMap<Double, List<Student>> classRankings() {
    NavigableMap<Double, List<Student>> ranked = new TreeMap<>(Comparator.reverseOrder());
    ...
}
```

Coding to the `NavigableMap` *interface* (not the `TreeMap` class) in the return type is itself a small but real lesson: callers only see "a sorted map," never anything TreeMap-specific, so the implementation could be swapped later without touching a single caller.

No `HashSet` for "unique course tracking" — and that's a deliberate choice, not an oversight

The brief mentions a `HashSet` for unique course tracking. Grep the codebase and you won't find one. `SubjectRepositoryImpl` explains why in its own comment:

```
// SubjectRepositoryImpl.java
/**
 * ... duplicate subject code (checked via the map key itself - a second HashSet would just
 * duplicate what the map's own keys already are) ...
 */
```

The `Map<String, Subject>`'s own key set **already is** the uniqueness guarantee — every key in a `Map` is unique by definition. Adding a parallel `HashSet<String>` just to "track uniqueness" would mean two data structures both claiming to represent the same invariant, with no mechanism forcing them to agree — a classic source of silent drift bugs (add to the map, forget to add to the set; or vice versa). **This is exactly the kind of question a reviewer will ask** — "where's your HashSet?" — and the honest answer ("I didn't need one, here's why") is a stronger answer than having added one just because the brief mentioned it.

Other collection choices worth knowing

- `CopyOnWriteArrayList` in `AuditTrail` — a `List` implementation that copies its entire backing array on every write, making reads (`getEntries()`) safe to call from any thread with **zero locking**, at the cost of an O(n) copy per write. Correct trade-off here: writes are already serialized through a single-thread executor (see §6), so writes are rare relative to potential concurrent reads.
- `ConcurrentHashMap` inside `LruCache` — a lock-striped hash map safe for concurrent reads *and* writes without external synchronization, unlike a plain `HashMap` (not thread-safe at all) or a `synchronized HashMap` / `Hashtable` (thread-safe but serializes *every* access behind one lock).
- Two-level storage in `GradeRepositoryImpl` : a `Grade[200]` array (the original source of truth for ordering and the "storage full" check) *plus* a `Map<String, LinkedList<Grade>>` secondary index keyed by student ID, so `findGradesByStudentId()` is O(1) + O(k) instead of an O(n) scan over every grade ever recorded.

---

4. File I/O and NIO.2

All new file-handling code uses **NIO.2** (`java.nio.file.Path` / `Files`), not the older `java.io.File` / `FileWriter` / `FileReader` . `FileExporter` (the one pre-existing v2 class that used to use `java.io.File`) was rewritten to NIO.2 as part of this work — the class Javadoc explains the migration and the one thing that deliberately did **not** change: the *string shape* of a returned file path stays `"<dir>/<filename>"` with a forward slash regardless of platform, since existing tests and console messages depend on that exact text — `Path.toString()` alone would use the OS-native separator, which is backslash on Windows.

Why NIO.2 over the old `java.io` API

- `Path` is a richer abstraction than a `File` string — it understands filesystem providers, supports `resolve()` / `getParent()` composition, and every `Files` method throws a specific checked `IOException` subtype (`NoSuchFileException` , `FileAlreadyExistsException` , ...) instead of `java.io` 's pattern of returning `false` / `null` on failure and leaving you to guess why.
- Stream-based reading: `Files.lines(path, charset)` returns a `Stream<String>` you can `.filter()` / `.map()` / `.toList()` directly — see `CSVParser.parse()` and both `*DataImporter` classes' `importCsv()` .

The three export/import formats, and what each one teaches

Format	Mechanism	Where
CSV	Manual line-by-line parsing (`String.split(",", -1)`), one record per row	`CSVParser`, `StudentDataExporter` / `GradeDataExporter`
JSON	Jackson's `ObjectMapper`, pretty-printed	`*DataExporter` / `*DataImporter`, `BackupServiceImpl`
Binary	Java's built-in `ObjectOutputStream` / `ObjectInputStream` serialization	`StudentDataExporter` / `GradeDataExporter`

`StudentRecord` / `GradeRecord` are plain Java **records** implementing `Serializable` — a record gets `equals()` / `hashCode()` / `toString()` for free (useful for round-trip test assertions like `assertEquals(original, imported)`), and Jackson can deserialize a record directly via its canonical constructor without any extra configuration.

`try-with-resources` everywhere a stream/writer is opened

Every file write/read that opens a `BufferedWriter` / `BufferedReader` / `ObjectOutputStream` uses `try-with-resources`, guaranteeing the resource is closed even if an exception is thrown mid-write:

```
// StudentDataExporter.java
public void exportJson(List<StudentRecord> students, Path path) {
    try (var writer = Files.newBufferedWriter(path, StandardCharsets.UTF_8)) {
        objectMapper.writerWithDefaultPrettyPrinter().writeValue(writer, students);
    } catch (IOException e) {
        throw new ExportException("Failed to export students as JSON: " + e.getMessage(), pa
    }
}
```

Notice the pattern repeated across **every** exporter/importer/backup class: NIO.2's `Files.*` methods throw checked `IOException`, and every one of these methods catches it and re-throws it wrapped in this app's own unchecked `ExportException` / `ImportException` / `BackupException`. That wrapping is the bridge between §5's two exception worlds.

---

5. Exceptions — Checked vs. Unchecked, and Why `BackupException` Is Different

The rule this codebase follows (and why)

Every custom exception in `main.exceptions` **except one** extends `ApplicationException`, which itself extends `RuntimeException` — i.e., every one of them is **unchecked**:

```
// ApplicationException.java
public abstract class ApplicationException extends RuntimeException {
    protected ApplicationException(String message) { super(message); }
    protected ApplicationException(String message, Throwable cause) { super(message, cause);
}
```

This is a deliberate architectural choice, not an oversight — the class's own Javadoc says why: it lets `ConsoleApp` catch **one type**, `ApplicationException`, as its final catch-all handler for every expected error condition, instead of forcing every layer between a thrown exception and `ConsoleApp` to declare `throws SpecificException` on every method signature along the way. A checked exception's cost is exactly that: it must be declared or caught at *every* intermediate call site, not just where it's ultimately handled. For a console app with one central error handler, that cost buys nothing.

Where the checked-vs-unchecked line actually gets drawn (Effective Java's rule of thumb)

- **Checked exception**: a *recoverable* condition where the caller is expected to catch it and do something genuinely different in response — not just log-and-rethrow.
- **Unchecked (`RuntimeException`)**: a programming error, or a condition the caller can't meaningfully recover from at that call site — appropriate to let bubble up to a shared handler.

`BackupException` — the one deliberate exception to the "everything unchecked" rule

```
// BackupException.java
public class BackupException extends Exception {   // note: NOT ApplicationException, NOT Ru
    private final BackupErrorCode errorCode;
    private final String filePath;
     ...
}
```

`BackupException` extends `Exception` directly — it is **statically unrelated** to `RuntimeException` / `ApplicationException`. (Proof, and a good thing to know for a review: an `instanceof` check against either type doesn't even *compile* against a `BackupException` reference — the compiler already knows the types share no relationship. See `BackupExceptionTest.isCheckedNotUncheckedTest()`.)

Why this one, specifically? Because a failed backup/restore genuinely has three *different* recovery paths, and the caller needs to know which one applies before deciding what to do:

`BackupErrorCode`	What it means	A caller might respond by...
`IO_FAILURE`	Couldn't read/write the file at all (disk full, permissions, missing path)	Check disk space, verify the path, retry
`CORRUPT_FILE`	The file exists and is readable, but isn't a valid backup	Try a different/earlier backup file
`VERSION_MISMATCH`	Valid backup, but from an incompatible format version	Use a compatible app version, or a newer backup

A single unchecked catch-all would blur these three into "backup failed, reason: message string," losing the compiler's ability to *force* every call site to at least think about which case applies. Because it's checked, `BackupService`'s two methods (`createBackup` / `restoreBackup`) both declare `throws BackupException` — and **every caller, everywhere, must either catch it or declare it themselves**, enforced at compile time, not just by convention:

```
// BackupService.java
public interface BackupService {
    void createBackup(Path path, List<StudentRecord> students, List<GradeRecord> grades) thr
    BackupPayload restoreBackup(Path path) throws BackupException;
}
```

`BackupAction` is the one place in the whole console layer that catches this checked exception directly (rather than letting a `RuntimeException` bubble to `ConsoleApp`'s catch-all), and it reports the specific error code to the user:

```
// BackupAction.java
} catch (BackupException e) {
    System.out.println("\n✗ ERROR: BackupException (" + e.getErrorCode() + ")");
    System.out.println("  " + e.getMessage());
}
```

"Exception signatures" — what that phrase actually means

A method's **exception signature** is the set of checked exceptions listed in its `throws` clause — part of its public contract, exactly like its parameter types. `void execute()` and `void execute() throws BackupException` are two *different* signatures; a caller of the second one **cannot compile** unless it either

catches `BackupException` or re-declares it in its own `throws` clause. That's the entire mechanical difference between checked and unchecked exceptions in one sentence: **unchecked exceptions are not part of a method's signature; checked ones are, and the compiler enforces it.**

The rest of the hierarchy (all unchecked, for contrast)

`StudentException`, `GradeException`, `SubjectException`, `*NotFoundException`, `*ValidationException`, `ImportException`, `ExportException`, `CSVImportException`, `InvalidGradeException` — each carries whatever specific recovery data its scenario needs (a student ID, a file path, an attempted grade value...) as a field with a getter, exactly like `BackupException` does, but none of them force a `throws` declaration anywhere, because `ConsoleApp` catches all of them as `ApplicationException`.

---

6. Multithreading & Concurrency

Every concurrent class deliberately demonstrates a **different** `java.util.concurrent` tool — this is explicit in the code's own comments, so a reviewer asking "why did you pick that one" has a direct answer for each:

`AuditTrail` — single-thread `ExecutorService`

```
ExecutorService executor = Executors.newSingleThreadExecutor( ... );
```

One background thread drains a queue of write tasks. Since only **one** thread ever writes, two concurrent callers of `append()` can never interleave or corrupt each other's entry — the executor itself is the serialization point, with no explicit lock needed. `append()` returns a `Future<Boolean>` so a caller (mainly tests) *can* wait for the write to land, but ordinary business code fires-and-forgets it, same as a logging call.

`ScheduledGpaRecalculationJob` — `ScheduledExecutorService`

```
executor.scheduleAtFixedRate(this::runOnce, 0, periodMillis, TimeUnit.MILLISECONDS);
```

A single-thread scheduled executor runs one task repeatedly on a fixed period (daily, by default). `scheduleAtFixedRate` is the right primitive here specifically because there's exactly one recurring job, and only one run should ever be in flight — a fixed-rate schedule with a single worker thread guarantees that.

`StatisticsDashboard` — a manual ticker thread plus a `ThreadPoolExecutor`

This one is intentionally different from the scheduled job, to show a second concurrency shape: a dedicated thread runs a `while (running) { sleep; submit(); }` loop (the "ticker"), and each tick's actual statistics computation is submitted to a `ThreadPoolExecutor` configured exactly like `Executors.newCachedThreadPool()` would build one — unbounded threads, a 60-second keep-alive, and a `SynchronousQueue` (a queue with **zero** capacity: a task can only be handed off directly to a waiting

thread, never buffered). The rationale, from the class's own Javadoc: the dashboard's per-tick work is "bursty," unlike the GPA job's one-task-on-a-fixed-schedule shape — a cached pool suits bursty, short-lived work; a single scheduled thread suits one steady recurring task.

`BatchReportService` — fixed-size `ExecutorService`, one task per student

```
Executors.newFixedThreadPool(threadCount)   // threadCount is bounded MIN_THREADS..MAX_THREA
```

Every student ID in a batch becomes its own `Future<StudentReportOutcome>`; the method blocks (`future.get()`) on each in turn, and one student's failure (unknown ID, disk error) is caught and recorded as *that student's own outcome* — it never aborts the rest of the batch. This is literally why `Future` / `ExecutorService.submit()` exists as opposed to just spawning raw `Thread`s: you get a handle to wait on and a place to catch that one task's exception without losing track of the others.

`LruCache<K, V>` — `ConcurrentHashMap` plus a per-entry `AtomicLong` clock

The textbook LRU cache uses a synchronized `LinkedHashMap` with access-order iteration. This one deliberately does *not* — it's backed by a lock-free `ConcurrentHashMap`, with recency tracked per entry via a shared `AtomicLong` "clock" that increments on every access. Eviction (when over capacity) scans the entry set for the *stale-longest* entry and removes it. The trade-off: no single lock ever serializes `get()` / `put()`, at the cost of a linear scan during eviction — worth it because reads (`get()`) vastly outnumber evictions in this app's usage pattern.

The shared lock: `GradeManager`'s `ReentrantReadWriteLock`

Three of the classes above (`StatisticsDashboard`, `ScheduledGpaRecalculationJob`, and any console action calling `addGrade()`) can run concurrently against the *same* grade data. `GradeManager` exposes one `ReadWriteLock`:

```
private final ReadWriteLock lock = new ReentrantReadWriteLock();

public <T> T readLocked(Supplier<T> read) {
    lock.readLock().lock();
    try { return read.get(); } finally { lock.readLock().unlock(); }
}
```

`addGrade()` takes the **write** lock; the dashboard's `refreshOnce()` and the scheduled job's `runOnce()` both read *through* `readLocked()`, taking the **read** lock. A `ReadWriteLock` allows many concurrent readers, but only one writer at a time and never a reader-during-a-write — exactly the guarantee needed here: a scheduled recalculation or a live dashboard tick must never observe a grade addition mid-write (a "torn read"), but multiple reads (two dashboard ticks, or a tick and a recalculation) don't need to block each other at all.

The two testing techniques worth knowing (used throughout this suite)

1. **Executor injection seams**: most concurrent classes have a second constructor/`start()` overload accepting an injectable `ExecutorService` / `ScheduledExecutorService` / `ThreadPoolExecutor`, defaulting the public no-arg version to build a real one. Tests substitute a Mockito mock to deterministically hit shutdown/timeout branches — Mockito's default return for an unstubbed `boolean` is `false`, which naturally drives an `if (!executor.awaitTermination( ... ))` timeout branch without an actual 30-second wait.
2. **Pre-interrupt technique**: calling `Thread.currentThread().interrupt()` *before* invoking a method that internally calls a blocking JDK method (`Future.get()`, `awaitTermination()`, `Thread.join()`) makes that call throw `InterruptedException` **immediately** — checking and clearing the calling thread's interrupt flag is part of those methods' documented contract. This is a clean, real (non-mocked), deterministic way to test an `InterruptedException` catch block.

---

7. Regex — Validation and Search

`ValidationPatterns` centralizes every `Pattern` used across the app (student/grade ID, email, phone — both local and international formats, course code, ISO date) so a format only needs to change in one place. `StudentValidator` / `GradeValidator` / `SubjectValidator` compile these once as `static final Pattern` fields (compiling a regex is relatively expensive; doing it once at class load, not per call, matters if validation runs in a hot path).

`StudentSearcher.searchByEmailPattern(String regex)` is the one place a regex is compiled **from user input** at runtime (`Pattern.compile(regex)`), not from a fixed constant — and it's the one place `PatternSyntaxException` (an unchecked exception the `Pattern` class itself throws for a malformed regex) is caught explicitly in the console layer, since it isn't part of this app's own `ApplicationException` hierarchy.

---

8. Anticipated Reviewer Questions

Grouped by topic, with the honest, code-grounded answer already worked out:

Collections

- *"Why LinkedHashMap instead of HashMap for your repositories?"* → Insertion-order iteration had to match the old array-based behavior exactly; a plain HashMap gives no ordering guarantee at all, which would have silently broken "get the first seeded student" call sites.
- *"Where's your HashSet for unique tracking?"* → Didn't need one — a `Map`'s key set already guarantees uniqueness; a parallel `HashSet` would just be a second place for the same invariant to drift out of sync.

- *"Why TreeMap for rankings instead of sorting a List?"* → The map *stays* sorted as new entries are added, rather than needing an explicit sort step called every time it's read; and coding to `NavigableMap` (the interface) instead of `TreeMap` (the class) means callers never depend on TreeMap specifically.

Exceptions

- *"Why is almost everything a RuntimeException?"* → So `ConsoleApp` can have one catch-all handler instead of every intermediate layer needing a `throws` declaration for exceptions it can't do anything about anyway.
- *"Then why does BackupException break that rule?"* → Because backup/restore failures genuinely need three different recovery strategies (retry, use a different file, upgrade), and a checked exception forces every caller to actually consider which applies, at compile time.
- *"Show me it's actually checked, not just named like one."* → `BackupException extends Exception` directly; `instanceof RuntimeException` against it won't even compile, since the compiler can prove the types are unrelated — see `BackupExceptionTest`.

Concurrency

- *"How do you prevent a torn read while a grade is being written?"* → `GradeManager`'s `ReentrantReadWriteLock`: `addGrade()` takes the write lock; the dashboard and scheduled job both read through `readLocked()`, taking the read lock. Many readers, one writer, never both. Cache: `LruCache` is backed by `ConcurrentHashMap` plus a per-entry `AtomicLong` access timestamp — no single lock serializes reads/writes; eviction scans for the stalest entry.
- *"Why a different Executor type for each concurrent class?"* → Each demonstrates a distinct shape of concurrent work: one steady recurring task → single-thread `ScheduledExecutorService`; bursty per-tick work → cached `ThreadPoolExecutor`; N independent one-shot tasks → fixed `ExecutorService` with one `Future` per task; strictly-serialized writes → single-thread `ExecutorService`.
- *"How did you test timeout/interrupt branches without waiting 30 seconds in every test run?"* → Executor-injection seams (substitute a mocked executor whose `awaitTermination` returns `false` or throws) and the pre-interrupt technique (interrupt the test thread before calling into a blocking method).

File I/O

- *"Why NIO.2 over java.io?"* → Richer `Path` API, specific checked `IOException` subtypes instead of `File`'s boolean/null failure signals, and direct `Stream<String>` support via `Files.lines()`.
- *"Why three export formats?"* → Each teaches a different serialization mechanism: CSV is manual text parsing; JSON is Jackson reflection-based (and works on records natively); binary is the JDK's own `ObjectOutputStream` graph serialization.

---

9. Study Plan — What To Learn Next, In Order

If you want to actually *own* this material rather than just defend it, work through these in order — each step points at the exact class in this repo that demonstrates it, so you can read the real code right after (or while) learning the concept:

1. **Collections & generics fundamentals** — the `Map` / `List` / `Set` / `Queue` interface hierarchy, what each concrete implementation trades off (ordering, null-handling, thread-safety, Big-O of `get` / `put` / `contains`). Read: `LinkedHashMap` vs `HashMap` vs `TreeMap` javadocs, then `StudentRepositoryImpl` / `GPACalculator.classRankings()`.
2. **Checked vs. unchecked exceptions, and *why* the distinction exists** — read Joshua Bloch's *Effective Java*, Item on exceptions (checked exceptions for recoverable conditions). Then read `ApplicationException.java`'s Javadoc, then `BackupException.java` and `BackupService.java` side by side.
3. **`try` / `catch` / `finally` / `try-with-resources`, and exception chaining** (`new ExportException(msg, cause)`) — read any `*DataExporter` / `*DataImporter` class top to bottom.
4. **Threads, `Runnable`, and why raw `Thread` isn't enough** — read `StatisticsDashboard`'s `tickLoop()` (a raw `Thread`) versus its executor (`ThreadPoolExecutor`), to see both used side by side in one class.
5. **The Executor framework**: `ExecutorService`, `ScheduledExecutorService`, `Future`, `shutdown()` / `awaitTermination()` / `shutdownNow()` — read all five `main.concurrent` classes in the order listed in §6 above; each adds one new piece.
6. **Locks and thread-safety**: `synchronized` vs `ReentrantLock` / `ReentrantReadWriteLock` vs lock-free structures (`ConcurrentHashMap`, `AtomicLong`, `CopyOnWriteArrayList`) — read `GradeManager`'s lock usage, then `LruCache`, then `AuditTrail`.
7. **NIO.2 file I/O**: `Path`, `Files`, and the `Stream<String>` returned by `Files.lines()` — read `CSVParser.parse()` end to end.
8. **Regex**: `Pattern` / `Matcher`, precompiling vs. compiling per call, `PatternSyntaxException` — read `ValidationPatterns.java`, then `StudentSearcher.searchByEmailPattern()`.
9. **Put it together**: trace one full request end to end — e.g. "record a grade while the dashboard is running" — from `RecordGradeAction` → `GradeManager.addGrade()` (write lock, cache invalidation, async audit append) → what the dashboard's next tick sees. That single trace touches almost everything in this document at once.